MaxAir Technical – Interfacing Tuya WiFi Switches

Introduction

The purpose of this guide is to describe the steps required to integrate Tuya WiFi switches/relays to MaxAir.

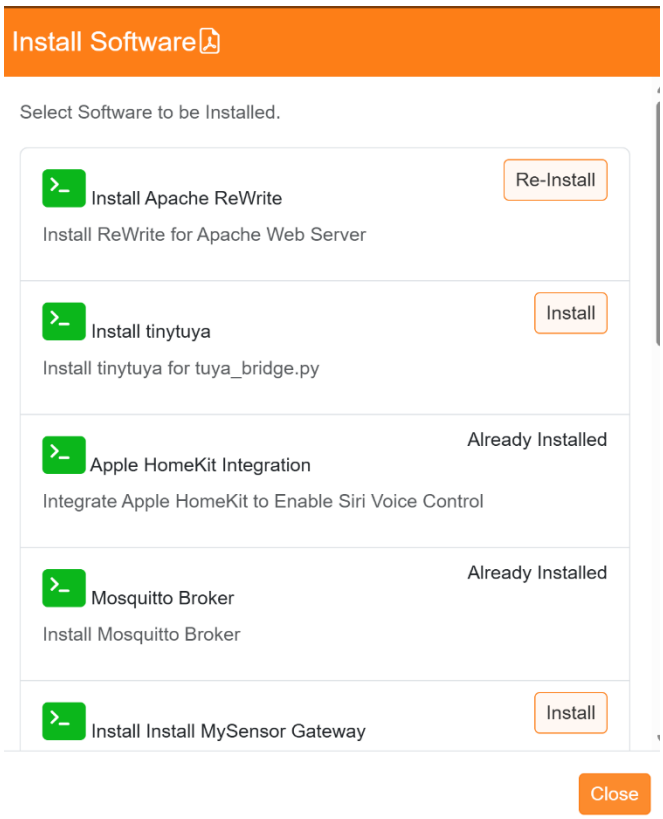
Integration with MaxAir

These switches/relays are the WiFi type rather than the ZigBee variety, they are normally controlled using the Tuya App and Cloud based service. The proposed solution uses a Python bridge script ('tuya_bridge.py') to enable communication with the Tuya device using MQTT.

In order that the Tuya device can both receive and report using MQTT, use is made of the Python module 'tinytuya', see https://github.com/jasonacox/tinytuya

The integration requires the generation of a 'keys' file, these keys are extracted using the instructions provided at the URL above, the file should be stored as '/home/maxair/devices.json'.

Python 'tinytuya' Library



To check if the 'Python 'tinytuya' module is installed use Settings/System Maintenance/Software Install option, if not shown as 'Already Installed' then click on the 'Install' button.

Can also be installed from the command line using:

```
sudo pip3 install tinytuya --break-system-packages
```

Running the Tuya bridge Script

The configuration file. '/home/maxair/devices.json', will for example be for a Tuya Mini Smart Switch with a name of 'WBP-1G and an ip address of 192.168.0.146:


```
[
  {
    "name": "WBP-1G",
    "id": "bf02e3173aecb1d2d7wcwm",
    "key": "zu$ZAHvDDetu:yMS",
    "mac": "20:f1:b2:78:6a:bc",
    "uuid": "f253ef6badf41e36",
    "category": "tdq",
    "product_name": "WBP-1G",
    "product_id": "zk2ktip9pzorzleb",
    "biz_type": 0,
    "model": "WBP-1G",
    "sub": false,
    "icon": "https://images.tuyaeu.com/smart/icon/ay15522876175557bVit/a65245324a45a0b71de6de3373072739.png",
    "mapping": {
      "1": {
        "code": "switch_1",
        "type": "Boolean",
        "values": {}
      },
      "9": {
        "code": "countdown_1",
        "type": "Integer",
        "values": {
          "unit": "s",
          "min": 0,
          "max": 86400,
          "scale": 0,
          "step": 1
        }
      },
      "20": {
        "code": "add_ele",
        "type": "Integer",
        "values": {
          "unit": "kW\u00b7h",
          "min": 0,
          "max": 50000,
          "scale": 3,
          "step": 100
        }
      },
      "21": {
        "code": "cur_current",
        "type": "Integer",
        "values": {
          "unit": "mA",
          "min": 0,
          "max": 30000,
          "scale": 0,
          "step": 1
        }
      },
      "22": {
        "code": "cur_power",
        "type": "Integer",
        "values": {
          "unit": "W",
          "min": 0,
```

```
        "max": 50000,
        "scale": 1,
        "step": 1
    }
},
"23": {
    "code": "cur_voltage",
    "type": "Integer",
    "values": {
        "unit": "V",
        "min": 0,
        "max": 5000,
        "scale": 1,
        "step": 1
    }
},
"24": {
    "code": "test_bit",
    "type": "Integer",
    "values": {
        "min": 0,
        "max": 5,
        "scale": 0,
        "step": 1
    }
},
"29": {
    "code": "fault",
    "type": "Bitmap",
    "values": {
        "label": [
            "ov_cr",
            "ov_vol",
            "ov_pwr",
            "ls_cr",
            "ls_vol",
            "ls_pow"
        ]
    }
},
"38": {
    "code": "relay_status",
    "type": "Enum",
    "values": {
        "range": [
            "0",
            "1",
            "2"
        ]
    }
},
"42": {
    "code": "random_time",
    "type": "String",
    "values": "{\"maxlen\":255}"
},
"43": {
    "code": "cycle_time",
    "type": "String",
    "values": "{\"maxlen\":255}"
},
"44": {
    "code": "switch_inching",
    "type": "String",
```

```
            "values": "{\"maxlen\":255}"
          },
          "47": {
            "code": "switch_type",
            "type": "Enum",
            "values": {
              "range": [
                "flip",
                "sync",
                "button"
              ]
            }
          }
        },
        "ip": "192.168.0.146",
        "version": "3.4"
    }
]
```


The script can be executed from the command line as 'python3 /var/www/cron/tuya_bridge.py'



The screen shows that the script is polling the Tuya device/s once every 45 seconds, is, in this case, subscribed to one topic – '**cmnd/car-charger/POWER**' and polls the tuya device to publish two MQTT messages:

car-charger: published {'POWER': 'OFF'}
car-charger: published {'1': False, '9': 0, '21': 0, '22': 0, '23': 2435, '24': 2, '29': 0, '38': 'memory', '42': '', '43': '', '44': 'AAAC', '47': 'flip'}

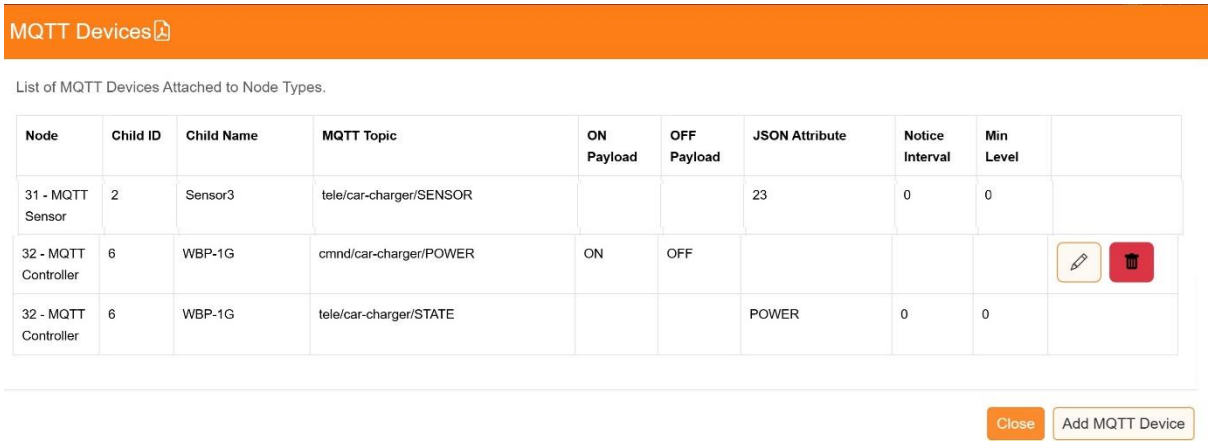
The first message is the device STATE and the second the device SENSOR response.

When MaxAir toggles the Tuya device state, as with any other MQTT type switch/relay, a MQTT message will be published and the tuya bridge script will action by using a 'tinytuya' function to set the state of the switch:

MQTT message received.
Topic: cmnd/car-charger/POWER
Message: ON
car-charger: set ON

MQTT Device Configuration

A connection to a Mosquitto broker must be configured using Settings/System Configuration/MQTT. The Tuya device/s are added using Settings/Device Configuration/MQTT Devices, an example is shown below, the 'JSON Attribute' for the SENSOR device uses the JSON Attribute obtained from the file . '/home/maxair/devices.json'. eg. 23 would return the 'cur_voltage' parameter or 22 would return the 'vur_power' parameter :



For further information on MaxAir MQTT configuration, please see the document – 'Setup Guide MQTT Devices'.

Enabling the Python TuyaBridge Script

Enable using Settings System Configuration/Jobs, click to check the enable box for job 'check_tuya_bridge' and then click on 'Apply'.

